Machine Learning – Refined Notes

ML Lifecycle

Machine learning follows a repeatable pipeline:

1. Problem Definition
2. Data Collection and Cleaning
3. Exploratory Data Analysis
4. Feature Engineering
5. Model Selection
6. Training
7. Evaluation
8. Hyperparameter Tuning

---

1. Problem Defintion

> translate a real-world into ML task

Inputs (Features)

- Examples:
  - Sleep hours ~ Normal(6, 1)
  - Workout duration ~ Normal(1.5, 0.25)
  - RPE ~ Uniform(1, 10)

Data Characteristics

- Feature distributions
  - Normal, Uniform...
- Data types:

    - Numerical
    - Categorical
    - Text
    - Image

Output (Target)

- Examples:
    - Fatigue level from sleep and workouts
- Types:
    - Regression → continuous value
    - Classification → discrete label

Objective

- Metrics:
    - $$Accuracy = \frac{TP + TN}{TP + TN + FN + FP}$$
        - how often is the model correct?
    - $$Precision = \frac{TP}{TP + FP}$$
        - of all positive predictions, how many were correct?
    - $$Recall = \frac{TP}{TP + FN}$$
        - of all actual positives, how many did we catch?
    - $$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$
        - harmonic mean of prediction and recall
- Metric choice depends on task type

Task type

- Regression: price, temperature, vehicle speed

    - MSE
    - RMSE
    - MAE
- Classification: spam/not, win/lose
    - accuracy
    - precision
    - recall
    - f1
- Clustering: customer segments
    - sillhouette score
    - inertia
- Ranking: recommendations
    - NDCG
    - MAP
    - MRR

---

2. Data Collection and Cleaning

Sources

- Kaggle
- Google Datasets

```
import requests

url = URL_DATASET
r = requests.get(url)

with open(DATASET, "wb") as f:
  f.write(r.content)
```

```
import pandas as pd

df = pdf.read_csv(DATASET_CSV)
```

Missing values

- Detect missing values: `df.isna().sum()`
- Drop rows or columns: `df.dropna()`
- Fill (mean, median, model-based)

```
df["col"].fillna(df["col"].mean(), inplace=True)
```

Outliers

- Detect outliers (IQR - Interquartile Range)

```
Q1 = df["salary"].quantile(0.25)
Q3 = df["salary"].quantile(0.75)
IQR = Q3 - Q1

outliers = df[(df["salary"] < Q1 - 1.5*IQR) |
              (df["salary"] > Q3 + 1.5*IQR)]
```

- Remove outliers

```
df = df[(df["salary"] >= Q1 - 1.5*IQR) &
        (df["salary"] <= Q3 + 1.5*IQR)]
```

Inconsistent formats

- Text standardization

```
df["city"] = df["city"].str.lower().str.strip()
```

- Data type conversions

```
# Convert to datetime
df["date"] = pd.to_datetime(df["date"])

# Convert to numeric (int/float)
df["value"] = pd.to_numeric(df["value"])

# One-hot encoding
encoded = pd.get_dummies(df, columns=["color"])

# Map to consistent boolean
df["flag"] = df["flag"].map({"Y": True, "N": False, "True": True, "False": False, "1": True, "0":

# Remove non-numeric characters and convert
df["price"] = df["price"].str.replace(r"\D", "", regex=True).astype(float)

# Extract numbers
df["weight"] = df["weight"].str.extract(r"(\d+)").astype(int)
```

Transform

- Scaling

> for regressions, SVMs, neural networks

$$x' = \frac{x - mean}{stddev}$$

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
df_scaled = scaler.fit_transform(df[["height", "weight"]])
```

- Normalization (min-max)

> in range of 0-1

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}}$$

```python
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()
df_norm = scaler.fit_transform(df[["height", "weight"]])
```

Feature Encoding

Categorical Features

- Integer encoding
  - red = 0
  - green = 1
  - blue = 2
  - for decision trees

```python
from sklearn.preprocessing import LabelEncoder
import pandas as pd

df = pd.DataFrame({
    "color": ["red", "green", "blue", "green", "red"]
})

encoder = LabelEncoder()
df["color_encoded"] = encoder.fit_transform(df["color"])

print(df)
print(encoder.classes_)

# color   color_encoded
# red     2
# green   1
# blue    0
```

- One-hot encoding
  - Target category = 1
  - Others = 0

Numerical Features

- Skewed features → log transform

$$x' = log(x + 1)$$

```
import numpy as np
import pandas as pd

df = pd.DataFrame({
    "income": [300, 500, 1000, 10000, 100000]
})

df["income_log"] = np.log1p(df["income"])
# log1p handles zeros safely
```

- Standardization:

$$z = \frac{x - mean}{std}$$

Text Encoding

- Bag of Words
  - Text -> vector of word frequencies
  - Word counts
  - Remove stop words

```python
from sklearn.feature_extraction.text import CountVectorizer

texts = [
    "I love machine learning",
    "machine learning is powerful",
    "I love powerful models"
]

vectorizer = CountVectorizer()
X = vectorizer.fit_transform(texts)

print(vectorizer.get_feature_names_out())
print(X.toarray())

# ['i', 'is', 'learning', 'love', 'machine', 'models', 'powerful']
# [
#  [1 0 1 1 1 0 0]
#  [0 1 1 0 1 0 1]
#  [1 0 0 1 0 1 1]
# ]
```

- Learned embeddings
  - Text → dense vectors
  - Integer encoding -> embedding

```
import tensorflow as tf
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

texts = [
    "I love machine learning",
    "machine learning is powerful",
    "I love powerful models"
]

tokenizer = Tokenizer()
tokenizer.fit_on_texts(texts)

sequences = tokenizer.texts_to_sequences(texts)
padded = pad_sequences(sequences)

print(tokenizer.word_index)
print(padded)

embedding_layer = tf.keras.layers.Embedding(
    input_dim=len(tokenizer.word_index) + 1,
    output_dim=8  # embedding size
)

embedded = embedding_layer(padded)
print(embedded.shape)
```

Image Encoding

- Images are tensors (3D: RGB)
- Techniques:
  - Flattening
  - Pixel normalization
  - Color space transforms
  - Edge detection
  - Deep learning representations

```
from tensorflow.keras.preprocessing import image
import numpy as np

img = image.load_img("cat.jpg", target_size=(224, 224))
img_array = image.img_to_array(img)

img_array = img_array / 255.0  # normalize
img_array = np.expand_dims(img_array, axis=0)

print(img_array.shape)

# (batch, height, width, channels)
# (1, 224, 224, 3)
```

- CNN-Based Image Encoding (Feature Extraction)

```
from tensorflow.keras.applications import ResNet50
from tensorflow.keras.applications.resnet50 import preprocess_input

model = ResNet50(weights="imagenet", include_top=False)

features = model.predict(preprocess_input(img_array))
print(features.shape)
```

3. Exploratory Data Analysis

> Understand patters and risks in the data

Visualizations

- Libraries

```
import matplotlib.pyplot as plt
import seaborn as sns

# Optional for interactive plots
import plotly.express as px
```

- Quick Overview

```
# Basic info
print(df.info())

# Summary statistics
print(df.describe())

# Check for missing values
print(df.isna().sum())
```

Numerical Features

- Distribution: skews, outliers, multi-modality

```
sns.histplot(df['sepal_length'], kde=True)
plt.title("Sepal Length Distribution")
plt.show()
```

- Boxplot: outliers

```
sns.boxplot(x=df['sepal_length'])
plt.title("Sepal Length Boxplot")
plt.show()
```

- Scatter Plot: Feature relationships, clusters, separability

```
sns.scatterplot(x='sepal_length', y='petal_length', hue='species', data=df)
plt.title("Sepal vs Petal Length by Species")
plt.show()
```

Categorical Features

> for classifiers, detect imbalance

```
sns.countplot(x='species', data=df)
plt.title("Class Distribution")
plt.show()
```

Correlations

> helps with feature selection, multicollinearity

- pearson
- Measures linear relationship between two continuous variables.
- Values range from -1 to 1.
  - 1 → perfect positive linear correlation
  - -1 → perfect negative linear correlation
  - 0 → no linear correlation
- Assumes variables are normally distributed.

```
import pandas as pd
from scipy.stats import pearsonr

# Sample data
df = pd.DataFrame({
    'x': [1, 2, 3, 4, 5],
    'y': [2, 4, 5, 4, 5]
})

# Pearson correlation
corr_matrix = df.corr()

# Using pandas
print(corr_matrix)

# Using scipy
corr, p_value = pearsonr(df['x'], df['y'])
print("Pearson correlation:", corr)
print("p-value:", p_value)

# Heatmap
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm')
plt.title("Feature Correlation Matrix")
plt.show()
```

- spearman
- Measures monotonic relationship (doesn't have to be linear).
- Works on ranks instead of raw values → robust to outliers.
- Values also range from -1 to 1.

```
# Using pandas
print(data.corr(method='spearman'))

# Using scipy
from scipy.stats import spearmanr
corr, p_value = spearmanr(data['x'], data['y'])
print("Spearman correlation:", corr)
print("p-value:", p_value)
```

- kendall
- Another rank-based correlation, more robust for small datasets or many ties.
- Measures how similar the ordering of the data is between two variables.

```
# Using pandas
print(data.corr(method='kendall'))

# Using scipy
from scipy.stats import kendalltau
corr, p_value = kendalltau(data['x'], data['y'])
print("Kendall's tau:", corr)
print("p-value:", p_value)
```

Pairwise relationships

> for clusters, separability, correlations

```
sns.pairplot(df, hue='species', diag_kind='kde')
plt.show()
```

Interactive visualizations

> for multidimensional inspection

```
fig = px.scatter(df, x='sepal_length', y='petal_length',
                 color='species', size='petal_width',
                 hover_data=['sepal_width'])
fig.show()
```

Distribution checks

- Histogram grid

```
num_cols = df.select_dtypes(include=np.number).columns

df[num_cols].hist(figsize=(12,8), bins=20)
plt.suptitle("Numerical Feature Distributions")
plt.show()
```

Correlation with Target

> Supervised Learning
> feature selection, most predictive features pop

```
# Encode target if categorical
df['species_encoded'] = df['species'].factorize()[0]

# Correlation with target
corr_target = df.corr()['species_encoded'].sort_values(ascending=False)
print(corr_target)
```

Clustering

> Unsupervised learning
> for identifying natural clusters, reducing dimensionality

```
from sklearn.decomposition import PCA

X = df[num_cols].values
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)

plt.scatter(X_pca[:,0], X_pca[:,1], c=df['species_encoded'])
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title("PCA Projection")
plt.show()
```

4. Feature Engineering

> Create informative inputs

- Too many features → overfitting

1. Feature Creation

- Aggregations

```
agg_user = df.groupby('user_id')['order_amount'].agg(total_spent='sum',
                                                     avg_spent='mean',
                                                     order_count='count').reset_index()
```

- Ratios

```
df['accuracy'] = df['hits_landed'] / df['hits_attempted']
```

- Domain-specific signals

```
# E-commerce example
df = pd.DataFrame({
    'price': [100, 200, 50, 300],
    'discount': [0, 20, 5, 50]
})

# Domain-specific signal: price after discount
df['final_price'] = df['price'] - df['discount']

# Another: discount percentage
df['discount_pct'] = df['discount'] / df['price']
```

- Polynomial features
  - $x \rightarrow [x, x^2, x^3]$
- Date-Time features
  - extract day from the DMY
- Text features

    - BoW, embeddings
- Image features
    - pixel intensities

2. Feature Transformation

> Transforming features to improve model performance.

- Scaling / Normalization
    - StandardScaler, MinMaxScaler, RobustScaler
    - Why: Ensures numeric features contribute equally, stabilizes training
- Log / Box-Cox / Yeo-Johnson Transformations
    - Reduce skew in distributions
    - Why: Helps linear models and gradient descent converge better
- Encoding Categorical Features
    - One-hot encoding, integer/label encoding, target encoding, embeddings
    - Why: Models can only process numeric inputs
- Binning / Discretization
    - Convert continuous variables to intervals
    - Example: Age group, income bracket
    - Why: Can capture non-linear patterns and reduce noise
- Handling Missing Values
    - Imputation: mean, median, mode, KNN, predictive models

3. Feature Selection

> Choosing the most useful features to reduce noise, dimensionality, and overfitting.

- Filter Methods
    - Correlation threshold, Chi-squared test, mutual information
    - Why: Remove irrelevant features before modeling
- Wrapper Methods
    - Recursive feature elimination, forward/backward selection
    - Why: Evaluate feature subsets using model performance
- Embedded Methods
    - Lasso, tree-based feature importance
    - Why: Select features during model training

4. Dimensionality reduction

> Reduce feature space while preserving information.

- Linear Methods
  - PCA, TruncatedSVD
  - Why: Reduce overfitting and computation, visualize data
- Non-linear Methods
  - t-SNE, UMAP, autoencoders
  - Why: Capture complex relationships for visualization or downstream models

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

# Sample data
df = pd.DataFrame({
    'feature1': [1,2,3,4,5],
    'feature2': [2,4,6,8,10],
    'feature3': [5,4,3,2,1]
})

# Standardize features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(df)

# Apply PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

# Convert back to DataFrame
df_pca = pd.DataFrame(X_pca, columns=['PC1', 'PC2'])
print(df_pca)
```

---

5. Model Selection

> Pick an algo suited to the problem and data size

Model Families

A model family is a space of functions mapping inputs to outputs.

Supervised Learning Models

Models that learn from labeled data (input → output)

Regression (Continuous target)

- Linear Regression
- Ridge Regression
- Lasso Regression
- Polynomial Regression
- Support Vector Regression (SVR)
- Decision Tree Regressor
- Random Forest Regressor
- Gradient Boosting Regressor
- Neural Networks (MLP, Deep Feedforward Networks)
- Bayesian Regression

Classification (Discrete target)

- Logistic Regression
- Decision Tree Classifier
- Random Forest Classifier
- Gradient Boosting Classifier (XGBoost, LightGBM, CatBoost)
- Support Vector Machine (SVM)
- k-Nearest Neighbors (k-NN)
- Naive Bayes (Gaussian, Multinomial)
- Neural Networks (MLP, CNN for images, RNN for sequences)
- Ensemble Classifiers (Voting, Stacking)

Unsupervised Learning Models

Models that learn patterns from unlabeled data

Clustering

- k-Means

- k-Medoids
- Hierarchical Clustering (Agglomerative, Divisive)
- Density-Based (DBSCAN, OPTICS)
- Gaussian Mixture Models (GMM)

Dimensionality Reduction / Feature Learning

- Principal Component Analysis (PCA)
- Linear Discriminant Analysis (LDA)
- Truncated SVD
- t-SNE
- UMAP
- Autoencoders (Vanilla, Variational)

Ensemble Learning Models

Combine multiple base models (base learners) to improve performance

Bagging (Reduce variance)

- trains models independently
- reduces variance
- Types
    - Random Forest
    - Bagged Decision Trees

Boosting (Reduce bias)

- trains models sequentially
- risk of overfitting
- Types
    - AdaBoost
    - Gradient Boosting
    - XGBoost
    - LightGBM
    - CatBoost

Stacking

- Meta-model combining multiple base learners

Voting

- Hard Voting Classifier
- Soft Voting Classifier

Support Vector Machines

- maximizes margins between classes

---

6. Training

> Learn parameters from data (weights)

Split data

> for evaluating generalization

- 80% training
  - 80% training
  - 20% validation
- 20% test

```
from sklearn.model_selection import train_test_split

data_tr, data_te, labels_tr, labels_te = train_test_split(
    data, labels, test_size=0.2, random_state=42
)

data_tr, data_val, labels_tr, labels_val = train_test_split(
    data_tr, labels_tr, test_size=0.2, random_state=42
)
```

Regression

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# Sample dataset
df = pd.DataFrame({
    'feature1': [1, 2, 3, 4, 5],
    'feature2': [5, 4, 3, 2, 1],
    'target': [2.2, 2.8, 3.6, 4.5, 5.1]
})

X = df[['feature1', 'feature2']]
y = df['target']

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train linear regression
model = LinearRegression()
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Evaluate
mse = mean_squared_error(y_test, y_pred)
print("Linear Regression MSE:", mse)
```

Classification

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# Iris dataset
from sklearn.datasets import load_iris
data = load_iris()
X = pd.DataFrame(data.data, columns=data.feature_names)
y = pd.Series(data.target)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train classifier
clf = RandomForestClassifier(n_estimators=100, random_state=42)
clf.fit(X_train, y_train)

# Predict
y_pred = clf.predict(X_test)

# Evaluate
accuracy = accuracy_score(y_test, y_pred)
print("Random Forest Accuracy:", accuracy)
```

Clustering

```
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

# Synthetic 2D data
import numpy as np
X = np.array([[1,2],[1,4],[1,0],[4,2],[4,4],[4,0]])

# Train k-Means
kmeans = KMeans(n_clusters=2, random_state=42)
kmeans.fit(X)

# Cluster labels
labels = kmeans.labels_
centers = kmeans.cluster_centers_

print("Cluster labels:", labels)
print("Cluster centers:\n", centers)

# Visualize
plt.scatter(X[:,0], X[:,1], c=labels)
plt.scatter(centers[:,0], centers[:,1], color='red', marker='x', s=100)
plt.title("k-Means Clustering")
plt.show()
```

Ensemble Learning

```
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.metrics import mean_absolute_error

# Sample dataset
X = pd.DataFrame({
    'feature1': [1,2,3,4,5,6,7,8],
    'feature2': [8,7,6,5,4,3,2,1]
})
y = pd.Series([3,3.5,4,4.5,5,5.5,6,6.5])

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

# Train model
gbr = GradientBoostingRegressor(n_estimators=100, learning_rate=0.1, random_state=42)
gbr.fit(X_train, y_train)

# Predict
y_pred = gbr.predict(X_test)

# Evaluate
mae = mean_absolute_error(y_test, y_pred)
print("Gradient Boosting MAE:", mae)
```

Optimize Loss Functions

Regression

- Mean Squared Error (MSE)
- Mean Absolute Error (MAE)
- Huber Loss

```python
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
import numpy as np
import pandas as pd

# Sample dataset
df = pd.DataFrame({
    'x1': [1,2,3,4,5],
    'x2': [5,4,3,2,1],
    'y': [2.1,2.9,3.5,4.1,5.0]
})

X = df[['x1','x2']]
y = df['y']

# Train-test split
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Linear Regression
model = LinearRegression()
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Loss (MSE)
mse = mean_squared_error(y_test, y_pred)
print("Mean Squared Error:", mse)

# Important parameters in regression models:
# - fit_intercept: whether to include bias term
# - normalize: scale features
# - n_jobs: parallel computation (for large datasets)
```

Classification

- Binary Cross-Entropy
- Categorical Cross-Entropy

- Hinge Loss

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import log_loss, accuracy_score
from sklearn.datasets import load_iris

# Dataset
data = load_iris()
X = data.data
y = data.target

# For binary demo, use only classes 0 and 1
X_bin = X[y!=2]
y_bin = y[y!=2]

X_train, X_test, y_train, y_test = train_test_split(X_bin, y_bin, test_size=0.2, random_state=42)

# Logistic Regression
clf = LogisticRegression(max_iter=200, solver='lbfgs')
clf.fit(X_train, y_train)

y_prob = clf.predict_proba(X_test)
y_pred = clf.predict(X_test)

# Loss
loss = log_loss(y_test, y_prob)
acc = accuracy_score(y_test, y_pred)
print("Log Loss:", loss)
print("Accuracy:", acc)

# Important parameters:
# - penalty: 'l1', 'l2', 'elasticnet', controls regularization
# - C: inverse of regularization strength
# - solver: optimization algorithm ('lbfgs', 'saga', 'liblinear')
# - max_iter: iterations for convergence
```

Clustering

```
from sklearn.cluster import KMeans
import numpy as np

# Synthetic data
X = np.array([[1,2],[1,4],[1,0],[4,2],[4,4],[4,0]])

# k-Means
kmeans = KMeans(n_clusters=2, init='k-means++', n_init=10, max_iter=300, random_state=42)
kmeans.fit(X)

labels = kmeans.labels_
centers = kmeans.cluster_centers_

# Loss = Inertia (sum of squared distances to cluster center)
print("Inertia (Loss):", kmeans.inertia_)

# Important parameters:
# - n_clusters: number of clusters
# - init: method to initialize centroids ('k-means++', 'random')
# - n_init: number of times the algorithm runs with different seeds
# - max_iter: iterations per run
```

Ensemble

- Gradient Boosting

```python
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.metrics import mean_absolute_error

# Sample dataset
X = np.array([[1,2],[2,1],[3,4],[4,3],[5,5]])
y = np.array([2.1,2.5,4.0,4.2,5.0])

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Gradient Boosting
gbr = GradientBoostingRegressor(
    n_estimators=100,      # number of trees
    learning_rate=0.1,     # step size for boosting
    max_depth=3,           # depth of each tree
    loss='ls',             # 'ls' = least squares, can also be 'lad', 'huber'
    random_state=42
)
gbr.fit(X_train, y_train)

y_pred = gbr.predict(X_test)
mae = mean_absolute_error(y_test, y_pred)
print("Gradient Boosting MAE:", mae)

# Important parameters:
# - n_estimators: number of boosting rounds
# - learning_rate: contribution of each tree
# - max_depth: tree complexity
# - loss: which loss function to optimize
```

---

7. Evaluation

> test on test data

- data leakage
- overfitting

Regression

```
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
import matplotlib.pyplot as plt

# Sample dataset
import pandas as pd
df = pd.DataFrame({
    'x1':[1,2,3,4,5,6],
    'x2':[6,5,4,3,2,1],
    'y':[2,3,4,5,6,7]
})

X = df[['x1','x2']]
y = df['y']

X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.33, random_state=42)
model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

# Metrics
mse = mean_squared_error(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print("MSE:", mse, "MAE:", mae, "R²:", r2)

# Visualization
plt.scatter(y_test, y_pred)
plt.xlabel("Actual")
plt.ylabel("Predicted")
plt.title("Regression: Actual vs Predicted")
plt.show()
```

Classification

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_mat

# Dataset
data = load_iris()
X = data.data
y = data.target

# Binary classification for demo
X_bin = X[y!=2]
y_bin = y[y!=2]

X_train, X_test, y_train, y_test = train_test_split(X_bin, y_bin, test_size=0.3, random_state=42)
clf = RandomForestClassifier(random_state=42)
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
y_prob = clf.predict_proba(X_test)[:,1]

# Metrics
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))
print("F1-score:", f1_score(y_test, y_pred))

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)

# ROC Curve
RocCurveDisplay.from_estimator(clf, X_test, y_test)
plt.show()
```

Clustering

```
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
import numpy as np

# Sample data
X = np.array([[1,2],[1,4],[1,0],[4,2],[4,4],[4,0]])

# Fit k-Means
kmeans = KMeans(n_clusters=2, random_state=42)
labels = kmeans.fit_predict(X)

# Metrics
inertia = kmeans.inertia_
sil_score = silhouette_score(X, labels)
print("Inertia:", inertia)
print("Silhouette Score:", sil_score)

# Visualization
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)
plt.scatter(X_pca[:,0], X_pca[:,1], c=labels)
plt.scatter(kmeans.cluster_centers_[:,0], kmeans.cluster_centers_[:,1], color='red', marker='x', s
plt.title("Clustering Visualization")
plt.show()
```

Ensemble

```
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.metrics import mean_squared_error, r2_score

# Sample regression data
X = np.array([[1,2],[2,1],[3,4],[4,3],[5,5]])
y = np.array([2,2.5,4,4.5,5])

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Gradient Boosting
gbr = GradientBoostingRegressor(n_estimators=50, learning_rate=0.1, max_depth=3, random_state=42)
gbr.fit(X_train, y_train)
y_pred = gbr.predict(X_test)

# Metrics
print("MSE:", mean_squared_error(y_test, y_pred))
print("R²:", r2_score(y_test, y_pred))

# Feature Importance
importances = gbr.feature_importances_
print("Feature Importances:", importances)
```

Metrics

Regression Metrics

Metrics for models predicting continuous values

- **Mean Squared Error (MSE)**
  Measures average squared difference between predicted and actual values; penalizes large errors.
- **Root Mean Squared Error (RMSE)**
  Square root of MSE; same units as target variable.
- **Mean Absolute Error (MAE)**
  Average absolute difference between predictions and actuals; less sensitive to outliers.
- **R² Score (Coefficient of Determination)**

Proportion of variance explained by the model; ranges 0–1 (1 = perfect fit).

- **Adjusted R^2**
  Adjusted for number of features; prevents overestimation when adding irrelevant predictors.
- **Mean Absolute Percentage Error (MAPE)**
  Average percentage error; useful for relative performance.
- **Median Absolute Error**
  Median of absolute errors; robust to outliers.

Classification Metrics

Metrics for models predicting discrete classes

- **Accuracy**
  Fraction of correct predictions.
- **Precision**
  True Positives / (True Positives + False Positives); how many predicted positives are correct.
- **Recall (Sensitivity, True Positive Rate)**
  True Positives / (True Positives + False Negatives); how many actual positives are detected.
- **F1 Score**
  Harmonic mean of Precision and Recall; balances the two metrics.
- **ROC-AUC**
  Area under the Receiver Operating Characteristic curve; measures trade-off between TPR and FPR.
- **PR-AUC (Precision-Recall AUC)**
  Useful for imbalanced datasets; measures area under precision-recall curve.
- **Log Loss / Cross-Entropy Loss**
  Penalizes incorrect predicted probabilities; used in probability-based classifiers.
- **Confusion Matrix**
  Tabular summary of TP, FP, FN, TN counts.

Clustering Metrics

Metrics for evaluating unsupervised models (no labels or optional labels)

- **Inertia (Within-Cluster Sum of Squares)**
  Measures compactness of clusters; lower is better.
- **Silhouette Score**
  Measures how similar a point is to its own cluster vs other clusters; ranges -1 to 1.
- **Davies-Bouldin Index**
  Measures cluster separation; lower is better.
- **Calinski-Harabasz Index**
  Ratio of between-cluster dispersion to within-cluster dispersion; higher is better.
- **Adjusted Rand Index (ARI)**
  Compares clustering to ground truth labels (if available); adjusted for chance.
- **Mutual Information Score**
  Measures agreement between true and predicted clusters.

Ensemble Model Metrics

Depends on the type of task (regression or classification); ensemble metrics are **same as base models**, but often include:

- **Out-of-Bag Error (Bagging models like Random Forest)**
  Estimates generalization error using samples not used in training individual trees.
- **Feature Importance**
  Measures relative contribution of each feature to predictions.
- **Cross-Validation Metrics**
  Average performance across folds (MSE, Accuracy, F1, etc.)
- **Learning Curves**
  Shows training vs validation error as ensemble grows; useful for diagnosing overfitting.
- **Ensemble-Specific Metrics**
  - For Gradient Boosting: Improvement in loss per iteration
  - For Stacking: Performance of meta-model vs base models

Confusion Matrix

Actual \ Predicted	Positive	Negative
Positive	TP	FN
Negative	FP	TN

8. Hyperparameter Tuning

> adjust training knobs

- Set before training
- Examples:
  - Learning rate
  - Depth
  - Regularization strength
- Tuned using:
  - Grid search
  - Random search

Regression

```
import pandas as pd
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_squared_error

# Sample dataset
df = pd.DataFrame({
    'x1': [1,2,3,4,5,6,7,8],
    'x2': [8,7,6,5,4,3,2,1],
    'y': [2,2.5,3,3.5,4,4.5,5,5.5]
})

X = df[['x1','x2']]
y = df['y']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

# Hyperparameter grid
param_grid = {
    'alpha': [0.01, 0.1, 1, 10, 100],  # Regularization strength
    'solver': ['auto', 'svd', 'cholesky']
}

ridge = Ridge()
grid_search = GridSearchCV(ridge, param_grid, cv=3, scoring='neg_mean_squared_error')
grid_search.fit(X_train, y_train)

print("Best Hyperparameters:", grid_search.best_params_)

y_pred = grid_search.predict(X_test)
mse = mean_squared_error(y_test, y_pred)
print("Test MSE:", mse)
```

Classification

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, RandomizedSearchCV
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
import numpy as np

# Dataset
data = load_iris()
X = data.data
y = data.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Hyperparameter distribution
param_dist = {
    'n_estimators': [50, 100, 200],
    'max_depth': [None, 2, 4, 6],
    'max_features': ['auto', 'sqrt', 'log2']
}

rf = RandomForestClassifier(random_state=42)
random_search = RandomizedSearchCV(rf, param_distributions=param_dist, n_iter=5, cv=3, scoring='ac
random_search.fit(X_train, y_train)

print("Best Hyperparameters:", random_search.best_params_)

y_pred = random_search.predict(X_test)
print("Test Accuracy:", accuracy_score(y_test, y_pred))
```

Clustering

```
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
import numpy as np

# Sample data
X = np.array([[1,2],[1,4],[1,0],[4,2],[4,4],[4,0]])

# Hyperparameter options
k_values = [2, 3, 4]
init_methods = ['k-means++', 'random']

best_score = -1
best_params = {}

for k in k_values:
    for init in init_methods:
        kmeans = KMeans(n_clusters=k, init=init, n_init=10, max_iter=300, random_state=42)
        labels = kmeans.fit_predict(X)
        score = silhouette_score(X, labels)
        if score > best_score:
            best_score = score
            best_params = {'n_clusters': k, 'init': init}

print("Best Hyperparameters:", best_params)
print("Best Silhouette Score:", best_score)
```

Ensemble

```
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import mean_absolute_error
import numpy as np

# Sample data
X = np.array([[1,2],[2,1],[3,4],[4,3],[5,5]])
y = np.array([2,2.5,4,4.5,5])

param_grid = {
    'n_estimators': [50, 100, 150],
    'learning_rate': [0.01, 0.05, 0.1],
    'max_depth': [2, 3, 4]
}

gbr = GradientBoostingRegressor(random_state=42)
grid_search = GridSearchCV(gbr, param_grid, cv=3, scoring='neg_mean_absolute_error')
grid_search.fit(X, y)

print("Best Hyperparameters:", grid_search.best_params_)

y_pred = grid_search.predict(X)
mae = mean_absolute_error(y, y_pred)
print("MAE:", mae)
```

Gradient Descent

- Iterative optimization
- Variants:
  - Batch
  - Stochastic
  - Mini-batch

Taxonomy

Bias vs Variance

- Bias → underfitting
- Variance → overfitting

Entropy

- Measure of uncertainty
- High entropy → unpredictable

Support Vector Machines (SVM)

- Finds optimal hyperplane
- Maximizes margin
- Support vectors define boundary

Epoch

- One full pass over training data

Margin

- $(w \cdot \varphi(x))$

Regularization

- Adds penalty to loss
- Improves generalization

Inductive Bias

- Assumptions enabling generalization

Linear vs Non-Linear

- Linear → straight boundary
- Non-linear → curved, expressive

Fitting Behavior

- Underfitting → high bias
- Overfitting → high variance